

숫자 야구 게임

2024.05.27.

디지털 시스템 설계 (CSED 273)

무은재학부 20230642 이채영

무은재학부 20230904 조민서

1. 구현하고자 하는 주제와 그 목적

본 프로젝트로 연구하고자 하는 주제는 숫자 야구 게임이다. 자릿수가 겹치지 않는 임의의 세 자리 난수를 생성하여 사용자가 숫자를 맞추는 장치이다. 게임이 시작될 때 정답에 해당하는 숫자를 시스템 내부에서 임의로 선정하고, 사용자가 숫자를 입력하면 숫자를 맞춘 결과를 문자로 출력한다. 문자는 **FPGA의 7-segment LED**를 통해 확인할 수 있다. 숫자는 맞지만 위치가 틀리면 볼(ball, B), 숫자와 위치 모두 맞으면 스트라이크(strike, S), 숫자와 위치가 전부 틀리면 아웃(out, OUT)이다. 숫자를 맞추고 결과를 확인하는 과정을 반복하여 숫자 세 자리를 모두 맞추면 승리하는 게임이다. 디지털시스템설계 과목을 수강하며 배운 내용을 토대로 **FSM**을 설계하고, **Verilog**와 **FPGA**를 통해 숫자 야구 게임을 구현하는 것이 목적이다.

2. 구현과 관련된 수업 내용 및 배경 지식

1) Karnaugh map

불 대수식을 단순화(simplification)하면 회로에 들어가는 와이어와 게이트의 수를 줄여, 소비 전력을 줄이고 작동 속도를 높일 수 있다. **Karnaugh map**은 불 대수식을 단순화하는 방법 중 하나이다. 입력변수와 출력을 도식화하여 1이 나오는 패턴을 찾아 사각형의 묶음으로 단순화한다. 이때 묶을 수 있는 경우가 다양하다면, 가장 많은 1을 포함하는 경우를 선택하여 단순화를 진행한다.

2) 1-Bit Magnitude Comparator

1-Bit Magnitude Comparator란, 서로 다른 두 개의 **1-Bit** 수(A, B)가 입력으로 주어졌을 때 둘의 대소 관계를 판별하여 알맞은 출력을 1로, 나머지 출력은 0으로 설정해 출력하는 회로를 의미한다. 입력값이 A, B로 주어지고 A와 B가 같은 수라고 가정할 경우, 출력 "**A>B**"는 0, 출력 "**A=B**"는 1, 출력 "**A<B**"는 0이 된다.

본 프로젝트에서는 "**A=B**"의 관계만 필요하므로 출력 "**A>B**"와 "**A<B**"는 생략하여 구현한다.

3) Decoder

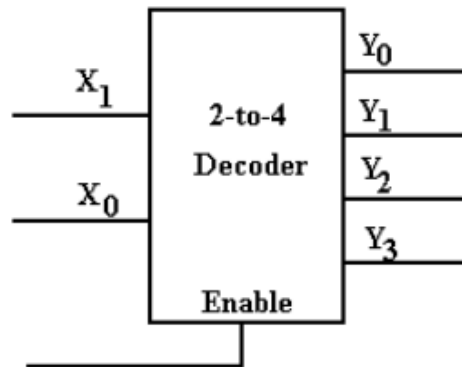
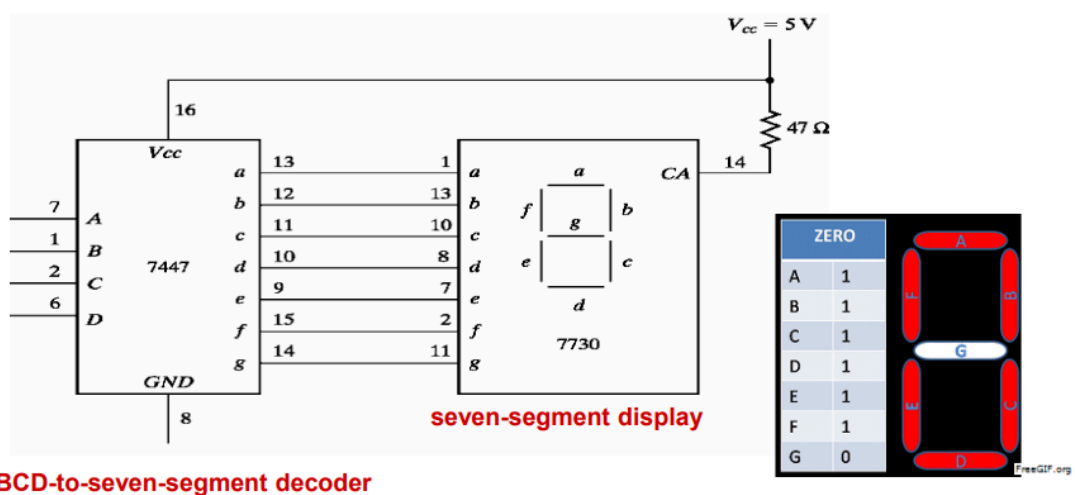


그림 1. 2-to-4 디코더

Decoder(디코더)는 n 개의 이진 정보 입력에서 최대 2^n 개의 고유 출력으로 변환하는 다중 입력, 다중 출력 회로이다. n 개의 이진 입력과 2^n 개의 서로 다른 출력을 가지는 경우, 각 출력이 곧 **minterm**이기 때문에 **minterm generator**라고도 한다. 실제 디코더 소자에는 n 개의 입력 뿐만 아니라 **EN**, 혹은 **Enable** 입력이 추가로 존재한다. **EN**은 디코더를 켜거나 끄기 위해서 사용한다. (**EN=1**인 경우 디코더는 정상적으로 작동하고, **EN=0**인 경우는 모든 입력에 대해 출력값이 같다)

디코더의 입력과 출력 특성을 표현할 때는 n -to- 2^n 과 k -of- 2^n 이라는 표현을 사용한다. n -to- 2^n 은 n 개의 입력을 받아 2^n 개의 출력을 가진다는 입력과 출력의 관계를 나타낸다. k -of- 2^n 은 2^n 개의 입력 중 k 개의 입력이 동시에 참이 된다는 출력의 특성을 나타낸다.

4) 7-segment display

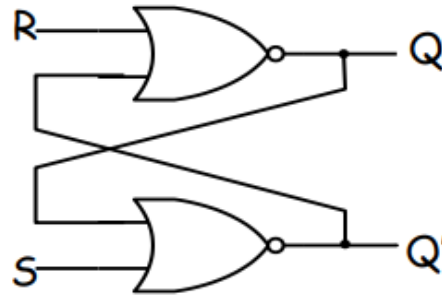


7-Segment display는 7개의 획으로 구성되어 있으며, 위와 아래의 사각형 모양으로 두 개의 가로 획, 두 개의 세로 획이 배치되어 있는 디스플레이이다. **7-segment display**는 일반적으로 숫자 또는 영문자를 표현하기 위해 사용된다. 본 프로젝트에서는 사용자가 입력한 숫자가 어느 정도 맞았는지 그 결과(**ball**, **strike**, **out**)를 표현하는데 사용된다.

5) SR latch

래치(latch)는 1비트의 정보를 보관 혹은 유지할 수 있는 회로이며, **sequential logic**의 기본 요소이다. **Computational logic**과는 다르게 이전 상태를 유지하여 저장할 수 있으며 각종 **counter circuit**, **register**, **RAM**등을 구성하는 기본 요소이자 **CPU**를 구성하는 밑바탕이 된다. **Latch**는 **Enable**에 의해 회로가 동작한다.

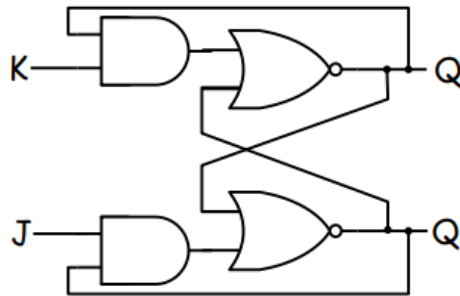
SR Latch에서 **S**는 **set**, **R**은 **reset**을 의미한다. 래치의 출력값인 **Q**와 **Q'**는 서로 **complement** 관계이므로, **Q=0**이면 **Q'=1**, **Q=1**이면 **Q'=0**이 된다.



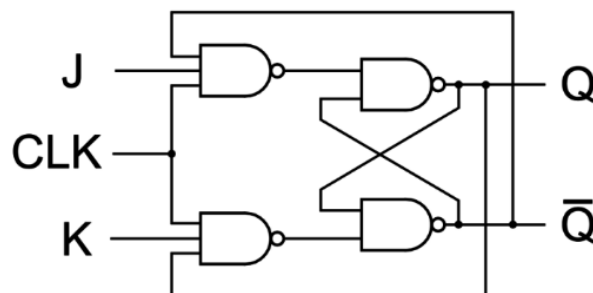
NOR 게이트로 구성된 **SR latch**는 **S=1, R=0**일 때는 **Set**, **S=0, R=1**일 때는 **Reset**이 동작된다. **S=0, R=0**이면 출력(**Q**) 상태가 이전과 같게 유지되고, **S=1, R=1**일 때는 두 출력 **Q, Q'**가 모두 **0**이 되기 때문에 사용하지 않는다.

6) JK Latch / JK Flip-Flop

JK Latch는 **SR Latch**에 추가적인 회로를 더해 **S=1, R=1**인 상태에서도 정상적으로 작동되도록 수정한 **Latch**이다. **JK Latch**에서는 **S=1, R=1**일 때 출력 결과가 반전된다.

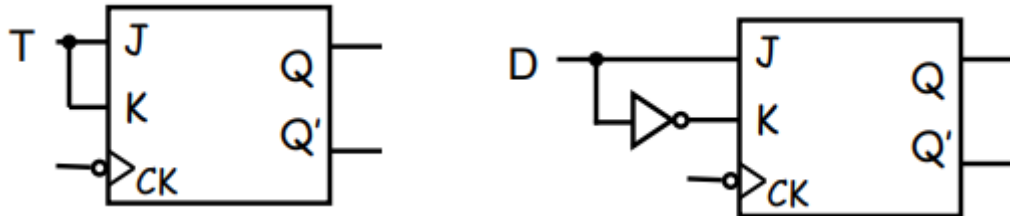


Latch는 입력이 바뀔 때 출력이 즉시 바뀌는 비동기 회로이다. 반면 **Flip-Flop**은 입력이 바뀌더라도 출력이 클럭(**CLK**)에 맞춰 반영되는 동기 회로이다. **JK Flip-Flop**은 클럭 신호를 추가로 받아 이에 맞춰 작동된다.



7) T Flip-Flop / D Flip-Flop

T(toggle) Flip-Flop과 D(delay) Flip-Flop은 JK Flip-Flop에서 J와 K 입력이 함께 묶여 있는 Flip-Flop이다. T FF는 입력값 T가 1일 때 기존의 출력값(Q)이 Toggle되며, D FF는 입력값(D)이 Delay 되어 출력값(Q)이 D의 값을 갖게 된다.

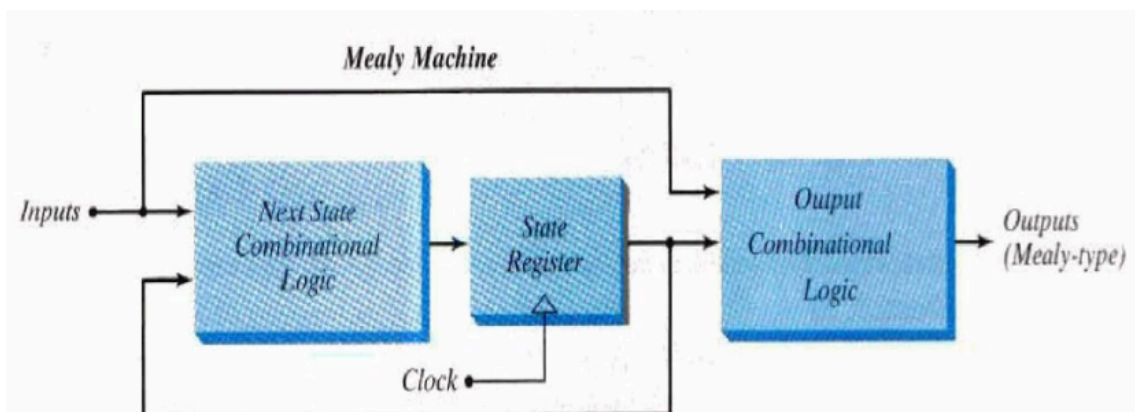
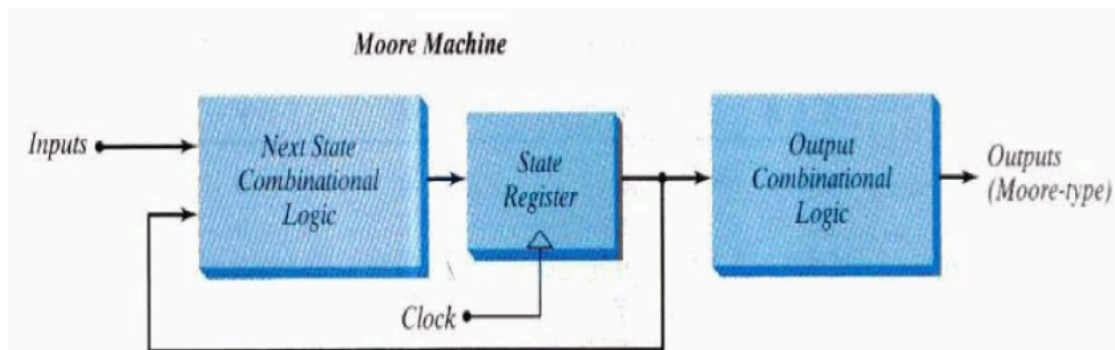


8) RAM

RAM은 사용자가 내용을 자유롭게 읽고, 쓰고, 지울 수 있는 기억 장치이다. 컴퓨터가 켜지는 순간부터 동작을 하는데 필요한 모든 내용이 전원이 켜져있는 내내 RAM에 저장된다. 본 프로젝트에서 RAM은 사용자가 입력한 정답을 저장하기 위해 사용된다.

9) FSM

FSM(Finite State Machine, 유한 상태 기계)은 state 개수가 유한한 sequential circuit이다. FSM은 Moore Machine, Mealy Machine으로 나뉘어진다. Moore Machine은 출력값이 현재 상태에 대한 함수이며, 활성화된 Clock edge와 동기화되게 상태 전이한다. Mealy Machine은 출력값이 현재 상태와 입력값에 대한 함수로, 입력값이 변화하자마자 출력값이 변화한다.



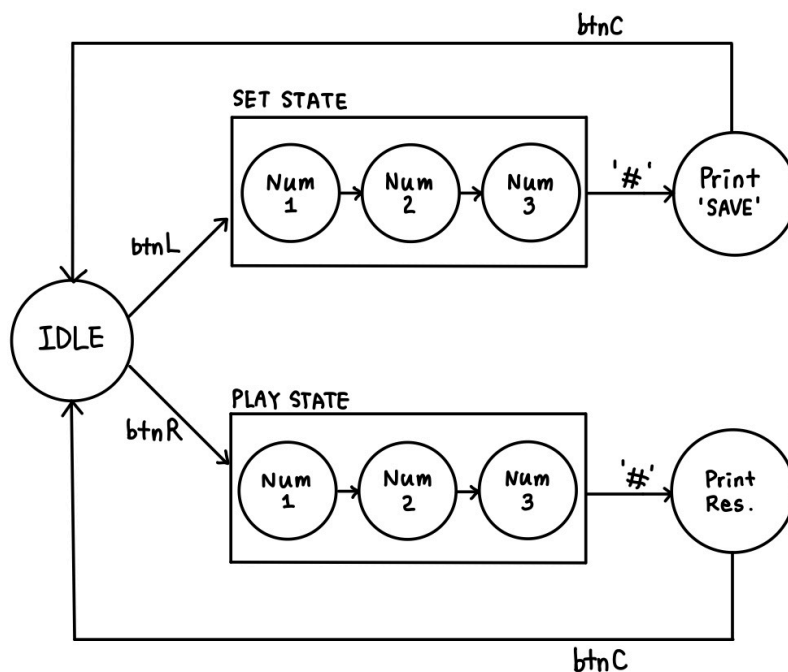
3. State transition diagram 및 State machine 설계

Device를 처음 Power On 했을 때는 IDLE state에 진입한다. 여기에서 FPGA 보드의 왼쪽 버튼을 누르면 Set state로, 오른쪽 버튼을 누르면 Play state로 넘어간다.

Set state는 사용자로부터 '정답'을 입력 받는다. 사용자는 키패드를 이용해 중복되는 수가 없는 세 자리 수를 입력한다. 이후 키패드의 '#' 버튼을 누르면 정답 출력 state에 진입해 숫자를 저장하고 7-segment module에 'SAVE'를 출력하여 저장되었음을 확인한다. FPGA의 가운데 버튼을 누르면 IDLE state로 돌아간다. Set state는 재진입할 수 있으며, 재진입할 경우 사용자가 입력한대로 정답에 저장된 값이 변경된다.

Play state는 사용자가 정답을 맞추는 단계이다. 사용자가 키패드를 이용해 세 자리 수를 입력한다. 숫자를 입력하고 키패드의 '#' 버튼을 누르면 결과 출력 state로 넘어간다. 결과 출력 state에서는 사용자 입력과 정답을 비교한 결과를 FPGA 보드의 7-segment module을 이용해 출력한다. 결과는 OUT, 1B, 2B, 3B, 1S, 2S, 1B1S, 2B1S, 3S 총 9가지가 있다.(결과 판단 기준은 아래 후술) 이후 FPGA의 중앙 버튼을 누르면 IDLE state로 돌아간다.

Set state와 Play State 모두 한 자리씩 숫자를 입력할 때마다 7-segment module에 현재 입력한 숫자를 출력한다.



구현하고자 하는 숫자 야구 게임의 State transition diagram은 위와 같다.

Play state에서 결과를 판단하는 기준은 아래와 같다.

사용자로부터 세 자리 수를 입력받고, 정답에 해당하는 수와 숫자 및 자리(위치)를 비교한다. 숫자와 자리가 모두 맞은 경우는 스트라이크, 숫자는 맞았지만 자리는 틀린

경우는 볼이다. 사용자 입력과 정답의 숫자가 모두 다른 경우는 아웃이다. 아웃이 아닌 경우 결과는 스트라이크와 볼의 개수로 나타낸다.

예를 들어 **Set state**에서 입력한 정답이 **784**일 때, **Play state**에서의 입력에 따른 결과는 아래와 같다.

사용자 입력	결과
784	3S
785	2S
793	1S
847	3B
846	2B
627	1B
874	2B1S
745	1B1S
231	OUT

4. 전체적인 동작 설명

프로그램이 시작되면 버튼 조작을 통해 정답 설정과 게임 플레이 중 하나를 선택한다. 정답 설정 선택 시 사용자는 키패드를 이용해 0~9로 이루어진 중복되지 않는 세 자리 수를 입력한다. 이후 버튼 조작을 통해 다시 처음 상태로 돌아온다. 다시 정답 설정을 선택한 경우 정답을 다시 입력하고 저장된 정답이 변경된다. 게임 플레이 선택 시 사용자는 세 자리 수를 입력하고 사용자가 입력한 숫자와 정답을 비교하여 볼, 스트라이크, 아웃을 판단한 결과가 출력된다. 이후 버튼 조작을 통해 다시 처음 상태로 돌아와 사용자 선택에 따라 위 과정이 반복된다.

5. 입출력 방법 및 동작 설명

입력은 키패드와 **FPGA**의 왼쪽, 오른쪽, 중앙 버튼을 이용한다.

키패드를 이용하여 정답 숫자와 사용자 입력 숫자, 입력 완료 신호(**#**)를 입력받는다.

FPGA의 세 버튼을 이용하여 초기 상태, 정답 설정, 게임 플레이의 각 상태로 전환한다.

출력의 경우 **FPGA**의 **7-segment** 모듈을 이용한다. 각 **state**별로 다음과 같이 출력한다.

IDLE - IDLE

SET - 사용자가 입력한 숫자 세 자리

정답 저장 - **SAVE**

PLAY - 사용자가 입력한 숫자 세 자리

결과 출력 - **OUT, 1B, 1S, 2B, 1B1S, 2S, 3B, 2B1S, 3S** 중 입력에 해당하는 결과

6. 필요한 부품 및 그 사용처

키패드 - 키패드를 통해 사용자에게 세 자리 입력과 입력 완료 신호(**#**)를 받는다.